VIEW

Projection, REPL, SDK, UI

화면을 중심으로 읽지 않고 동일한 conversation truth의 서로 다른 소비자 표현으로 읽는다

> **UI는 핵심 상태가 아니다. 그러나 입력·취소·지연·가시성을 조정하기 때문에 핵심 불변조건을 쉽게 침범할 수 있다.**

대상 snapshot
v2.1.88 / 2ca5ddabfed5

설명 구조
조건 · 이론 · 구현 · 반례

제3자 복원 소스에 대한 교육용 분석이다. 코드 관찰, 이론적 비유, 개선 제안을 구분하며 누락 모듈을 추정해 채우지 않는다.

같은 turn도 소비자마다 다른 모습이 필요하다

REPL, SDK, transcript가 왜 같은 배열을 공유하면 안 되는가?

성립 조건

REPL은 progress와 interactive control을, SDK는 stable event를, transcript는 resume fidelity를 요구한다.

이론 렌즈

projection은 한 source model에서 목적별 view를 파생하며 canonical state와 수명을 구분한다.

코드의 구체화

QueryEngine/query event가 REPL rendering, SDK result, session record로 각각 변환된다.

깨지는 지점

UI 편의를 위해 제거한 event가 transcript에도 사라지거나 SDK normalization이 core state를 mutate하면 drift가 생긴다.

해석 surface 차이는 formatting이 아니라 fidelity·latency·durability policy 차이다.

근거와 연결 src/QueryEngine.ts · src/screens/REPL.tsx · artifacts/knowledge/operations/knowledge-projection-and-contextualization.md

정식 상태는 가장 상세한 화면이 아니다

> 어떤 state가 다음 transition을 결정하는가?

후보	장점	canonical로 삼을 때 위험
React state	즉시 interaction	render lifecycle·ephemeral
yielded events	시간 순서	consumer 중단 시 불완전
query internal state	transition fidelity	process-local
transcript	durable	repair/format lag
SDK result	stable public view	의도적 정보 축약

판정 기준

다음 core transition을 결정하는 owner와 crash 후 복원 source는 같을 수도 다를 수도 있으며, 그 관계가 명시되어야 한다.

추출된 engine은 migration seam이다

> wrapper가 존재한다는 사실이 ownership 이전을 의미하는가?

성립 조건

headless/SDK와 future REPL이 공통 conversation lifecycle을 필요로 한다.

이론 렌즈

strangler-style extraction은 old entrypoint를 유지한 채 새 owner로 책임을 옮긴다.

코드의 구체화

QueryEngine 주석은 lifecycle/session ownership과 ask에서의 추출 목적을 직접 밝힌다.

깨지는 지점

REPL에 queryGuard, prompt restore, abort orchestration이 남아 있으면 ownership이 중복된다.

해석 새 class의 존재보다 어떤 invariant의 마지막 writer가 어디인지가 migration 진척을 말해 준다.

근거와 연결 src/QueryEngine.ts · src/screens/REPL.tsx

render와 commit 사이에 core effect를 두지 않는다

> React의 render model이 agent runtime에 주는 제약은 무엇인가?

성립 조건

render는 state의 UI description을 계산하며 다시 실행될 수 있다.

이론 렌즈

React owner 문서는 trigger→render→commit을 구분한다.

코드의 구체화

Ink components는 query state를 화면 요소로 투영하고 hooks가 input/render coordination을 맡는다.

깨지는 지점

render path에서 tool effect나 durable write를 시작하면 재-render와 UI lifecycle이 core semantics를 바꾼다.

해석

UI component는 observation을 그려야 하며 effect authority와 canonical transition은 engine boundary에 남아야 한다.

근거와 연결 React render and commit · src/screens/REPL.tsx

terminal UI는 DOM과 다른 물리적 제약을 가진다

> 같은 React가 왜 다른 운영 문제를 만드는가?

terminal 조건	구조적 영향	실패 형태
ANSI diff output	render batching 필요	flicker/jank
TTY 여부	interactive capability 변화	CI/piped output 오작동
alternate screen/scrollback	history visibility 정책	출력 소실
폭·unicode	layout 측정	wrap/정렬 오류
stdin raw mode	input ownership	shortcut 충돌

Ink는 component model을 제공하지만 terminal의 stateful output과 lifecycle은 별도 adapter 책임이다.

근거와 연결 Ink owner repository · src/ink/reconciler.ts

보이는 진행과 정식 결과를 구분한다

progress event를 history에 모두 저장해야 하는가?

성립 조건

긴 tool은 사용자가 멈추지 않도록 중간 관찰을 제공해야 한다.

이론 렌즈

projection은 목적에 맞게 detail을 선택하되 source truth를 바꾸지 않는다.

코드의 구체화

tool progress/final result가 같은 stream을 공유하는 hack이 명시되어 있다.

깨지는 지점

progress를 model history에 넣으면 noise와 token cost가 증가하고, 아예 버리면 운영 진단이 어려워진다.

해석 progress는 UI/telemetry projection이며 terminal observation과 identity·durability를 분리해야 한다.

근거와 연결 src/services/tools/toolExecution.ts

generation number는 stale continuation을 격리한다

왜 boolean `isRunning`으로 부족한가?

성립 조건

cancel 직후 새 query가 시작되면 이전 async finally가 나중에 실행될 수 있다.

이론 렌즈

generation token은 continuation이 어느 epoch에 속하는지 확인한다.

코드의 구체화

REPL queryGuard가 current generation과 callback/finally의 generation을 비교한다.

깨지는 지점

old finally가 prompt/input state를 복구하면 새 turn의 UI를 덮는다.

해석 generation guard는 UI trick이 아니라 overlapping lifecycle의 ownership fence다.

근거와 연결 src/screens/REPL.tsx

prompt field는 단순 문자열 state가 아니다

> 입력 중 text와 제출된 command의 경계는 어디인가?

interactive surface는 draft text, submitted input, queued command, restored-on-error prompt를 구분해야 한다. 이 상태를 query engine message와 합치면 cancel/resubmit race에서 무엇이 이미 durable한지 모호해진다.

구조적 귀결	draft ownership은 REPL adapter에, accepted input identity와 turn admission은 conversation owner에 둔다.

근거와 연결 src/screens/REPL.tsx · src/utils/processUserInput/processUserInput.ts

SDK는 headless UI가 아니라 protocol consumer다

> 화면을 없애면 어떤 계약이 더 엄격해지는가?

interactive REPL	SDK/headless	공통 core
human confirmation UI	callback/policy port	permission decision semantics
incremental render	typed event stream	event identity/order
prompt restore	caller retry data	terminal reason
terminal width/TTY	transport serialization	message/tool contracts

SDK는 internal object를 노출하는 shortcut이 아니라, UI 없이도 lifecycle을 완결할 수 있는 stable projection이 필요하다.

검색 가능한 기록과 재생 가능한 기록은 다르다

> UI search용 text가 resume source가 될 수 있는가?

성립 조건

search는 사람이 찾기 쉬운 flattening을 원하고 resume는 typed event fidelity를 원한다.

이론 렌즈

같은 source에서 파생된 projection도 목적과 loss budget이 다르다.

코드의 구체화

Tool contract에 transcript/search 관련 presentation surface가 effect/result와 함께 존재한다.

깨지는 지점

검색 index를 canonical record로 재사용하면 id·variant·metadata가 손실된다.

해석	human-readable projection과 replayable journal을 분리하고 derivation을 연결해야 한다.

근거와 연결 src/Tool.ts · src/utils/sessionStorage.ts

여러 projection은 결국 같은 사건으로 합류해야 한다

> 동일성을 무엇으로 판단할까?

identity	연결 대상	검출 가능한 drift
queryChainId	parent/subturn	인과 단절
request/attempt id	fallback streams	중복/누락 attempt
tool_use id	proposal/result	orphan pairing
session/message id	journal/resume	duplicate history
generation id	REPL continuation	stale UI mutation

관측 원칙 timestamp proximity만으로 같은 사건을 추정하지 않고 명시 identity로 join한다.

UI latency는 core ordering을 바꾸라는 압력이 된다

> 반응성을 위해 무엇을 늦추고 무엇을 먼저 해야 하는가?

선택	얻는 것	지는 것
user record 선쓰기	kill 후 resume	초기 latency/incomplete turn
render batching	terminal 안정성	progress freshness
observer clone	instrumentation 격리	copy cost
stream raw buffer 최적화	CPU/memory	복잡한 assembly
tool parallelism	turn latency	ordering/reconciliation 부담

성능은 독립 concern이 아니라 durability·fidelity·concurrency와 tradeoff point를 만든다.

terminal에서 보이는 것과 전달되는 것은 다르다

> 색과 animation만으로 상태를 표현하면 무엇이 손실되는가?

permission, running, failed, interrupted 상태는 색상 외 label과 terminal text로도 구분되어야 SDK log·screen reader·plain output에서 의미가 남는다. UI의 시각적 강조는 protocol variant를 대신할 수 없다.

근거와 연결 local information design knowledge

projection 경계는 import rule로 고정할 수 있다

> 새 구조가 다시 결합되는 것을 어떻게 막는가?

fitness	막는 회귀
engine core → React/Ink 금지	presentation이 state owner가 됨
presenter → effect call 금지	render 재실행이 effect 발생
journal record → UI type 금지	durability가 surface lifecycle에 결합
CoreEvent exhaustive mapping	SDK/REPL 중 한쪽 event 누락
projection mutation 금지	observer가 canonical input 변경

실습: 두 surface의 허용 차이를 명시한다

> REPL과 SDK trace는 어디까지 같아야 하는가?

01	**fixture**	text success, tool progress, deny, abort, fallback, resume
02	**core trace**	proposal/effect/observation/terminal identity를 비교
03	**REPL-only**	draft, key input, visual batch를 허용
04	**SDK-only**	serialized callback/result shape를 허용
05	**failure**	core event 누락 또는 서로 다른 durable history

화면은 truth를 보여 주지만 truth를 소유하지 않는다

이 권에서 남겨야 할 한 문장은 무엇인가?

> **좋은 UI 분리는 React 파일을 옮기는 일이 아니라, interactive lifecycle이 core history와 effect authority를 바꾸지 못하게 하는 일이다.**

REPL과 SDK는 같은 conversation engine의 소비자여야 하지만 동일한 projection일 필요는 없다. 차이는 명시적이고 testable해야 한다.